Day 1 – Set 1: Retail Order Desk

orderId	customerId	amount	createdAt
656788	3679	227.05	2026-10-17
878279	1731	609.04	2026-10-22
946348	4433	886.15	2026-10-21
465955	4615	949.63	2026-10-05
915922	4314	2914.24	2026-10-18

Task 1: Linear and binary search

```
import csv, time
data = [int(r["orderId"]) for r in csv.DictReader(open("orders.csv"))]
data2 = sorted(data)
x = data2[-1]

def linear(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1

def binary(a, x):
    l, h = 0, len(a) - 1
    while l <= h:
        m = (l + h) // 2
        if a[m] == x: return m
        if a[m] < x: l = m + 1
        else: h = m - 1
    return -1

t = time.perf_counter(); linear(data, x); print("Linear:", time.perf_counter() - t)
t = time.perf_counter(); binary(data2, x); print("Binary:", time.perf_counter() - t)
```

Output

```
Linear: 0.00036120800000105646
Binary: 5.909999998721105e-06
```

Task 2: Bubble sort and merge sort

```
import csv, time
a = [float(r["amount"]) for r in csv.DictReader(open("orders.csv"))][:10000]

def bubble(a):
    a = a[:]
    for i in range(len(a)):
        for j in range(len(a) - i - 1):
            if a[j] < a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

def merge(a):
    if len(a) < 2: return a
    l, r = merge(a[:len(a)//2]), merge(a[len(a)//2:])
    res = []
    while l and r:
```

```
        res.append(l.pop(0) if l[0] > r[0] else r.pop(0))
    return res + l + r

t = time.perf_counter(); bubble(a); print("Bubble:", time.perf_counter() - t)
t = time.perf_counter(); merge(a); print("Merge:", time.perf_counter() - t)
t = time.perf_counter(); sorted(a, reverse=True); print("Built-in:", time.perf_counter() - t)
```

Output

```
Bubble: 4.686296691999999
Merge: 0.10423479099999611
Built-in: 0.0012389769999998634
```

Task 3: Duplicate orders

```
import csv
seen = set()
count = 0
for r in csv.DictReader(open("orders.csv")):
    key = (r["customerId"], r["amount"], r["createdAt"][:16])
    if key in seen:
        count += 1
    seen.add(key)
print("Duplicates:", count)
```

Output

```
Duplicates: 25
```

Task 4: Own Stack and Queue

```
from collections import deque

class Stack:
    def __init__(self): self.s = []
    def push(self, x): self.s.append(x)
    def pop(self): return self.s.pop()

class Queue:
    def __init__(self): self.q = []
    def add(self, x): self.q.append(x)
    def remove(self): return self.q.pop(0)

st = Stack()
st.push("edit A"); st.push("edit B"); st.push("edit C")
print("Undo:", st.pop())

q = Queue()
q.add(301); q.add(302); q.add(303)
print("Dispatch:", q.remove())

d = deque([301, 302, 303])  # built-in
print("Built-in dispatch:", d.popleft())
```

Output

```
Undo: edit C
Dispatch: 301
Built-in dispatch: 301
```

Task 5: Greedy and DP

```
def greedy(amt, coins):
    n = 0
```

```
    for c in sorted(coins, reverse=True):
        n += amt // c
        amt %= c
    return n

def dp(amt, coins):
    d = [0] + [999] * amt
    for i in range(1, amt + 1):
        for c in coins:
            if c <= i:
                d[i] = min(d[i], d[i - c] + 1)
    return d[amt]

print("Refund 1043:", greedy(1043, [500, 200, 100, 50, 20, 10, 5, 2, 1]))
print("Coins 1,5,6,9 and amount 11 -> Greedy:", greedy(11, [1, 5, 6, 9]), "DP:", dp(11, [1, 5
```

Output

```
Refund 1043: 6
Coins 1,5,6,9 and amount 11 -> Greedy: 3 DP: 2
```